Design and Analysis of Algorithms CSEL30503MJ

Module - 2

Greedy Method

Content

Introduction

Knapsack Problem,

Job Sequencing,

Optimal Merge Patterns,

Huffman coding,

Minimal Spanning Trees.

Suggestive Questions

Introduction: Greedy Method

The **Greedy Method** is a powerful algorithmic paradigm used to solve optimization problems. The core idea is to build a solution by making a sequence of choices. At each step, the algorithm makes the locally optimal choice—the one that looks best at the current moment—in the hope that this will lead to a globally optimal solution. A greedy algorithm typically has two properties:

Greedy Choice Property: A globally optimal solution can be reached by making a locally optimal choice.

Optimal Substructure Property: An optimal solution to the problem contains within its optimal solutions to its subproblems.

It's crucial to understand that not all optimization problems can be solved with a greedy approach. It only works if these two properties hold true.

The Knapsack Problem (Fractional)

Problem: Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given capacity and the total value is as large as possible.

Greedy Approach (Fractional Knapsack): This is the version where you can take fractions of an item. The greedy strategy is to **prioritize items with the highest value-to-weight ratio**.

Algorithm:

1. Calculate the value-to-weight ratio (vi/wi) for each item.
2. Sort the items in descending order based on this ratio.
3. Iterate through the sorted items. For each item:
    I. If the knapsack has enough capacity, take the whole item.
    II. If not, take a fraction of the item to fill the remaining capacity.

Example: The Knapsack Problem (Fractional)

- **Items:**
  - Item 1: v1=60,w1=10
  - Item 2: v2=100,w2=20
  - Item 3: v3=120,w3=30
- **Knapsack Capacity:** W=50

Solution:

1. **Calculate Ratios:**
   1. Item 1: 60/10=6
   2. Item 2: 100/20=5
   3. Item 3: 120/30=4
2. **Sort Items by Ratio (descending):** Item 1, Item 2, Item 3.
3. **Fill Knapsack:**
   1. **Take Item 1:** Capacity left = 50−10=40. Total value = 60.
   2. **Take Item 2:** Capacity left = 40−20=20. Total value = 60+100=160.
   3. **Take Item 3:** The remaining capacity is 20, but Item 3 has a weight of 30. We take a fraction of Item 3. Fraction = 20/30=2/3.
      1. Value added = (2/3)×120=80.
      2. Total value = 160+80=240.

Conclusion: The maximum value is 240.

Job Sequencing with Deadlines

Problem: Given a set of jobs, each with a deadline and a profit, find a sequence of jobs to perform on a single machine such that the total profit is maximized. A job can only be completed by its deadline.

Greedy Approach: The greedy strategy is to **schedule jobs with the highest profit first**.

Algorithm:

1. Sort the jobs in descending order of their profit.
2. Initialize an array of time slots (from 1 to the maximum deadline) to be empty.
3. Iterate through the sorted jobs. For each job:
    1. Find the latest possible available time slot (from its deadline down to 1) where it can be scheduled.
    2. If such a slot exists, place the job in that slot.

Example: Job Sequencing with Deadlines

- **Jobs:**
  - J1: d1=4,p1=20
  - J2: d2=1,p2=10
  - J3: d3=1,p3=40
  - J4: d4=1,p4=30
  - J5: d5=3,p5=15
- **Max Deadline:** 4

Solution:

1. **Sort by Profit (descending):** J3, J4, J1, J5, J2.
2. **Schedule Jobs:**
   1. **J3 (d=1,p=40):** Schedule at time slot 1.
      Schedule: [J3, _, _, _]
   2. **J4 (d=1,p=30):** Deadline is 1, but slot 1 is taken. Cannot schedule.
   3. **J1 (d=4,p=20):** Latest free slot is 4. Schedule at time 4.
      Schedule: [J3, _, _, J1]
   4. **J5 (d=3,p=15):** Latest free slot is 3. Schedule at time 3.
      Schedule: [J3, _, J5, J1]
   5. **J2 (d=1,p=10):** Deadline is 1, but slot 1 is taken. Cannot schedule.

Conclusion: The optimal sequence is J3, J5, J1, with a maximum profit of 40+15+20=75.

Optimal Merge Patterns

Problem: Given a set of sorted files of different sizes, find the minimum cost to merge them into a single sorted file. The cost of merging two files is the sum of their sizes.

Greedy Approach: The greedy strategy is to **repeatedly merge the two smallest files** until only one file remains. This is implemented using a min-heap data structure.

Algorithm:

1. Create a min-heap with the sizes of all the files.
2. While there is more than one file in the heap:
    1. Extract the two smallest file sizes, s1 and s2.
    2. Merge them, with a cost of s1+s2.
    3. Insert the new file size (s1+s2) back into the heap.
3. The total cost is the sum of all merge costs.

Example: Optimal Merge Patterns

- **File Sizes:** S=[20,30,10,5]

Solution:

1. **Min-Heap:** [5,10,20,30]
2. **Merge 1:** Merge 5 and 10.
    1. **Cost:** 5+10=15.
    2. **New Heap:** [15,20,30]
3. **Merge 2:** Merge 15 and 20.
    1. **Cost:** 15+20=35.
    2. **New Heap:** [30,35]
4. **Merge 3:** Merge 30 and 35.
    1. **Cost:** 30+35=65.
    2. **New Heap:** [65]

Conclusion: The total cost is 15+35+65=115.

Huffman Coding

Problem: Given a set of characters and their frequencies, find an optimal prefix code to represent them. A prefix code ensures that no code word is a prefix of another, allowing for unique decoding.

Greedy Approach: The greedy strategy is to **build a binary tree from the bottom up by repeatedly merging the two nodes with the lowest frequencies**.

Algorithm:

1. Create a min-heap of nodes, one for each character, with the frequency as the key.
2. While there is more than one node in the heap:
    1. Extract the two nodes with the minimum frequencies, n1 and n2.
    2. Create a new internal node nnew with a frequency equal to the sum of n1 and n2.
    3. Set n1 and n2 as the children of nnew (e.g., n1 is the left child, n2 is the right).
    4. Insert nnew back into the min-heap.
3. The final node remaining in the heap is the root of the Huffman tree. Traverse the tree to assign binary codes (e.g., 0 for left, 1 for right).

Example: Huffman Coding

- **Characters and Frequencies:**
  - 'a': 5, 'b': 9, 'c': 12, 'd': 13, 'e': 16, 'f': 45

Solution:

1. **Initial Nodes:** (5)a, (9)b, (12)c, (13)d, (16)e, (45)f
2. **Merge 1:** Merge 'a' (5) and 'b' (9).
   1. New node: (14)ab.
   2. Nodes: (12)c, (13)d, (14)ab, (16)e, (45)f
3. **Merge 2:** Merge 'c' (12) and 'd' (13).
   1. New node: (25)cd.
   2. Nodes: (14)ab, (16)e, (25)cd, (45)f
4. **Merge 3:** Merge 'ab' (14) and 'e' (16).
   1. New node: (30)abe.
   2. Nodes: (25)cd, (30)abe, (45)f
5. **Merge 4:** Merge 'cd' (25) and 'abe' (30).
   1. New node: (55)cdabe.
   2. Nodes: (45)f, (55)cdabe
6. **Final Merge:** Merge 'f' (45) and 'cdabe' (55).
   1. New node: (100)fcdabe. This is the root.

Huffman Codes:

- **'f': 0**
- **'c': 100**
- **'d': 101**
- **'a': 1100**
- **'b': 1101**
- **'e': 111**

Minimal Spanning Trees (MST)

Problem: Given a connected, undirected, weighted graph, find a subgraph that is a tree and connects all the vertices together, with the minimum possible total edge weight.

Greedy Approach: Both Prim's and Kruskal's algorithms are classic greedy approaches to finding an MST.

a) Prim's Algorithm

Strategy: Grow the MST from an arbitrary starting vertex. At each step, add the lowest-weight edge that connects a vertex already in the tree to a vertex not yet in the tree. **Example:**

- **Graph:** A graph with vertices A, B, C, D and edges: (A,B,2), (A,C,3), (B,C,1), (B,D,4), (C,D,5).

Minimal Spanning Trees (MST)

- **Solution:**

1. **Start at A:** The edges from A are (A,B,2) and (A,C,3). Choose (A,B,2).
    1. **MST Edges:** {(A,B,2)}
2. **Add from {A,B}:** The available edges are (A,C,3) and (B,C,1), (B,D,4). Choose (B,C,1) as it's the minimum.
    1. **MST Edges:** {(A,B,2), (B,C,1)}
3. **Add from {A,B,C}:** The available edges are (A,C,3), (B,D,4), (C,D,5). The minimum weight edge is (B,D,4).
    1. **MST Edges:** {(A,B,2), (B,C,1), (B,D,4)} **Total Cost:** 2+1+4=7.

Minimal Spanning Trees (MST): Kruskal's Algorithm

Strategy: Build the MST by adding edges in increasing order of weight. An edge is added only if it does not form a cycle with the edges already included in the MST. **Example:** (Same graph) **Solution:**

1. **Sort Edges by Weight:** (B,C,1), (A,B,2), (A,C,3), (B,D,4), (C,D,5)
2. **Add (B,C,1):** No cycle.
    1. **MST Edges:** {(B,C,1)}
3. **Add (A,B,2):** No cycle.
    1. **MST Edges:** {(B,C,1), (A,B,2)}
4. **Add (A,C,3):** This forms a cycle (A-B-C-A). Discard.
5. **Add (B,D,4):** No cycle.
    1. **MST Edges:** {(B,C,1), (A,B,2), (B,D,4)}
6. **Add (C,D,5):** This forms a cycle (C-B-D-C). Discard. **Total Cost:** 1+2+4=7.

Suggestive Questions

1. What are the two key properties that a problem must satisfy to be solvable by a greedy algorithm?
2. For the Fractional Knapsack problem, what is the greedy criterion for selecting items?
3. In the context of Huffman coding, what is a prefix code, and why is it important?
4. What is the primary difference between how Prim's and Kruskal's algorithms build a Minimal Spanning Tree?
5. For Job Sequencing with Deadlines, is it ever beneficial to schedule a job before its latest possible time slot?
6. A set of files has sizes [20, 15, 30, 10, 5]. Trace the greedy algorithm for finding the optimal merge pattern and calculate the total cost.
7. Consider a set of jobs: J1(p=100, d=2), J2(p=10, d=1), J3(p=15, d=2), J4(p=27, d=1). Using the greedy approach, find the optimal sequence of jobs and the maximum profit.
8. Prove why the greedy strategy for the Fractional Knapsack problem is optimal. You can use a proof by contradiction.

Suggestive Questions

8. Given the following graph, find the Minimal Spanning Tree using both Prim's and Kruskal's algorithms. Show the edges added at each step, and state the total weight of the MST.
   - Graph Edges and Weights: (A,B,4), (A,C,5), (B,C,6), (B,D,7), (C,D,2), (C,E,8), (D,E,3).
9. A message contains the following characters and their frequencies:

Character	Frequency
C	**32**
A	**25**
L	**15**
P	**10**
E	**9**
F	**9**
R	**7**

(a) Construct the Huffman tree and determine the Huffman codes for each character.

(b) Calculate the total number of bits required to encode the original message using the generated Huffman codes.